Beyond the Canvas: Efficient dLLMs via Self-Guided CoT Compression and Suffix Sparsification

Anonymous Author(s)
Affiliation
Address
email

Abstract

Diffusion large language models (dLLMs) have emerged as a promising alternative to autoregressive generation. Existing acceleration methods mainly improve per-step decoding parallelism, but largely overlook redundancy at the trajectory level. dLLMs iteratively unmask tokens on a fixed-length masked sequence (canvas), which is typically set relatively large to accommodate unpredictable Chain-of-Thought (CoT) generation. We observe that such a design can lead to two key problems that affect the efficiency of dLLM reasoning: *(i)* the redundancy in reasoning CoT induced by an oversized canvas; and *(ii)* the redundancy in suffix tokens as context. To alleviate these problems, we propose Beyond-the-Canvas dLLM (BC-dLLM), a framework with two complementary strategies: **S**elf-**G**uided CoT **C**ompression (SGC) and **P**rogress-**A**ware **S**uffix **S**parsification (PASS). Specifically, SGC strategy aims to decouple the reasoning length from the canvas size, enabling the model to generate compact CoTs on large canvases. SGC uses the dLLM itself to construct variable-length CoT supervision data and applies conditional supervised fine-tuning to internalize this compact reasoning behavior on large canvases. The design of the PASS strategy is based on our insight that information gain is typically concentrated in suffix tokens adjacent to the current decoding block and diminishes as reasoning progresses. Hence, PASS shrinks the suffix window as the reasoning progress for efficient computation in each step on the large canvas. Experiments on four reasoning benchmarks and multiple base models show that our method significantly reduces decoding steps while maintaining comparable performance to baselines. When applied to LLaDA-8B-Instruct, our method achieves a $10.1\times$ latency speedup on the GSM8K dataset while maintaining comparable performance.

1 Introduction

Diffusion large language models (dLLMs) [36, 54, 50] have recently emerged as a promising alternative to autoregressive (AR) generation [49, 1]. dLLMs operate on a fixed-length masked sequence, referred to as the *canvas*, and iteratively unmask multiple tokens per step with bidirectional attention. This formulation enables parallel decoding, random-order generation, and global context modeling, which promise substantial inference speedups. However, realizing this potential in practice has proven to be difficult. While closed-source dLLMs such as Mercury [26] and Seed Diffusion [42] report impressive throughput and competitive quality, open-source dLLMs still lag behind and, in many cases, run even slower than their AR counterparts of comparable scale. For example, LLaDA [36] still requires decoding steps roughly proportional to the sequence length to maintain quality, largely offsetting its theoretical parallelism advantage. Closing this efficiency gap is therefore crucial for making open-source dLLMs practical alternatives to AR LLMs.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Existing works can be broadly characterized as accelerating dLLMs by optimizing each decoding step. One line of work reduces per-step computation: Fast-dLLM [46] introduces a block-wise approximate KV-cache, while Sparse-dLLM [41] and ES-dLLM [55] skip unimportant tokens or layers in the forward pass. Another line of work increases per-step parallelism through improved schedulers, including confidence-aware unmasking [46, 52] and history-aware decoding [34, 25], and semi-autoregressive decoding [36, 46, 6], where multiple tokens are generated in parallel at each step while preserving left-to-right order across blocks. Despite these improvements, existing efforts largely overlook the inefficiency problem in the overall reasoning trajectory. Common applications of dLLMs require an oversized predefined canvas to accommodate the variable-length reasoning trajectories that dLLMs can produce. We reveal that this design exposes two key problems causing inefficiency: *(i)* **redundancy in reasoning CoT induced by the oversized canvas** (Sec. 3.2), where dLLMs produce unnecessarily verbose reasoning even on simple problems; and *(ii)* **redundancy in suffix tokens as context** (Sec. 3.3), where every decoding step processes the entire remaining canvas although most suffix tokens contribute progressively less information during decoding.

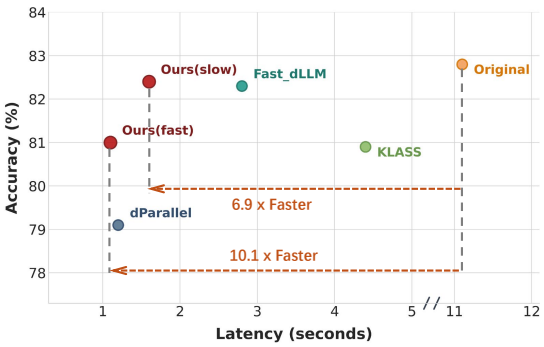

Figure 1: Performance and speed comparison between our method and SOTA methods on the GSM8K dataset. By setting corresponding conditional prompts, we can obtain results under slow and fast-thinking modes.

To address the above two problems, we propose Beyond-the-Canvas dLLM (BC-dLLM), a framework with two complementary strategies: **S**elf-**G**uided CoT **C**ompression (SGC) and **P**rogress-**A**ware **S**uffix **S**parsification (PASS). SGC tackles the first problem by decoupling CoT length from canvas size, enabling dLLMs to generate more compact reasoning paths on oversized canvases. The core idea is to leverage the dLLM itself as an internal compressor to construct variable-length CoT supervision data. During data construction, we first elicit comprehensive reasoning trajectories (*slow-thinking* mode) on a large canvas, and regenerate the later segments of these trajectories on a narrowed canvas, yielding more compact CoTs (*fast-thinking* mode). We then apply supervised fine-tuning (SFT) with conditional prompts on these CoT data to internalize compact reasoning capability on a large canvas. PASS is motivated by the observation that suffix information gain is typically concentrated near the current decoding block and diminishes as decoding progresses. It uses the predicted probability of the `<EOS>` token over the suffix tokens as an estimate of decoding progress, and dynamically shrinks the suffix window size as the decoding process, thereby cutting per-step computation on large canvases. By integrating SGC and PASS, BC-dLLM mitigates canvas-induced inefficiency and accelerates reasoning without major performance compromise as shown in Fig. 1. In fast-thinking mode, it achieves a $10.1\times$ latency speedup on GSM8K with LLaDA-8B-Instruct while maintaining competitive performance.

In summary, the contributions of this work are as follows:

- We identify two key bottlenecks in current dLLM acceleration frameworks induced by oversized canvases: redundancy in reasoning CoT due to the unnecessary verbosity for filling a large canvas, and redundancy in suffix tokens as context.
- We propose Beyond-the-Canvas dLLM (BC-dLLM), which integrates two complementary strategies: self-guided CoT compression (SGC) to decouple CoT length from canvas size, and progress-aware suffix sparsification (PASS) to progressively shrink the suffix window during decoding.
- We conduct extensive experiments to validate the effectiveness of BC-dLLM on different reasoning benchmarks and base dLLMs. Our method achieves substantial inference speedups while maintaining reasoning performance comparable to that of standard dLLMs.

2 Related Work

Diffusion Large Language Models. dLLMs [16, 36, 54, 50, 28, 19, 14] have emerged as a promising alternative to autoregressive LLMs [49, 1, 27, 43, 31]. They can be broadly categorized as continu-

ous [14, 28] and discrete methods [19, 36, 50]. Continuous dLLMs represented by Diffusion-LM [28] and CDCD [14] perform denoising in continuous embedding space and then project intermediate states back to tokens. Due to mismatches between continuous embeddings and discrete tokens, they often lead to unstable token predictions. In contrast, discrete dLLMs directly operate on a categorical token space. There is no continuous-to-discrete mismatch during generation. Representative discrete models include the LLaDA series [36, 54, 51], trained from scratch with forward masking and reverse unmasking, and the Dream family [50, 48], which adapts pretrained autoregressive LLMs to a discrete diffusion framework. Besides, there are also hybrid designs that combine autoregressive inter-block generation with parallel intra-block generation, such as SDAR [10] and Block Diffusion [3]. Here we focus on discrete dLLMs [36, 50] due to superior performance and growing research popularity.

Chain-of-Thought (CoT) Compression. CoT substantially improves complex reasoning in AR LLMs [45, 17, 18, 22], but long intermediate traces introduce considerable latency. To address this problem, prior works have explored CoT compression [24, 47, 29, 32], including RL-based approaches with explicit length regularization [32, 2, 29] and SFT-based approaches that distill shorter yet effective reasoning trajectories from curated supervision data [47, 24, 13]. Generally, RL-based methods are typically more costly to train, while SFT-based methods rely on obtaining high-quality curated trajectories. For example, TokenSkip [47], an SFT-based method, prunes less important tokens based on semantic contribution. In this work, we identify an entanglement between canvas size and CoT length in dLLMs, which induces redundant reasoning. Here we focus on SFT-based CoT compression for dLLMs and propose leveraging the dLLM itself as the CoT compressor.

Inference Acceleration for dLLMs. dLLMs still suffer from high inference latency due to iterative unmasking [23, 53, 15] and limited compatibility with the KV-cache mechanism used in AR models [40, 39]. Recent acceleration methods mainly follow two directions. The first improves token-level parallelism via better sampling schedulers, including confidence-aware unmasking such as Fast-dLLM [46] and Dimple [52], and history-aware decoding such as DInfer [34] and KLASS [25]. The second [46, 41, 55, 33, 8] reduces per-step computation. Specifically, Fast-dLLM [46] introduces a block-wise approximate KV-cache. Most relevant to our method, dPad [8] employs a fixed sliding window for suffix tokens. In contrast, our method explicitly models the block-wise decay of suffix-token contributions during decoding and adaptively allocates the suffix attention budget across decoding steps via reliable progress estimation, improving efficiency while preserving reasoning quality.

3 Method

In this section, we first introduce the preliminaries for the training and the inference process of dLLMs. We then present our proposed Self-Guided CoT Compression (SGC) and Progress-Aware Suffix Sparsification (PASS) strategies in detail.

3.1 Preliminaries

Masked Diffusion Large Language Models. Discrete dLLMs [4] have emerged as a promising non-autoregressive paradigm for natural language generation. As a representative variant, masked dLLMs model the generation as a probabilistic process comprising forward masking corruption and reverse denoising. The forward process gradually corrupts a clean token sequence $\mathbf{x}_0$ of length L into a noisy sequence $\mathbf{x}_t$ at continuous time level $t \in [0, 1]$:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \prod_{i=1}^{L} \Big[(1-t)\, \delta(x_t^i = x_0^i) + t\, \delta(x_t^i = [\texttt{MASK}]) \Big], \tag{1}$$

where x_t^i denotes the token at the i-th position, and $[\texttt{MASK}]$ is a special masking token. The reverse process is parameterized by a neural network p_θ, which attempts to recover the clean sequence $\mathbf{x}_0$ from the masked sequence $\mathbf{x}_t$ by jointly predicting all masked tokens under the assumption of conditional independence. In recent works [37, 4], the simplified objective of the model is to minimize the negative log-likelihood restricted only to the masked positions. This objective serves as a surrogate upper bound for the model's true negative log-likelihood:

$$\mathcal{L}(\theta) = -\mathbb{E}_{t, \mathbf{x}_0, \mathbf{x}_t} \left[\frac{1}{t} \sum_{i=1}^{L} \mathbf{1}[x_t^i = [\texttt{MASK}]] \log p_\theta(x_0^i \mid \mathbf{x}_t) \right], \tag{2}$$

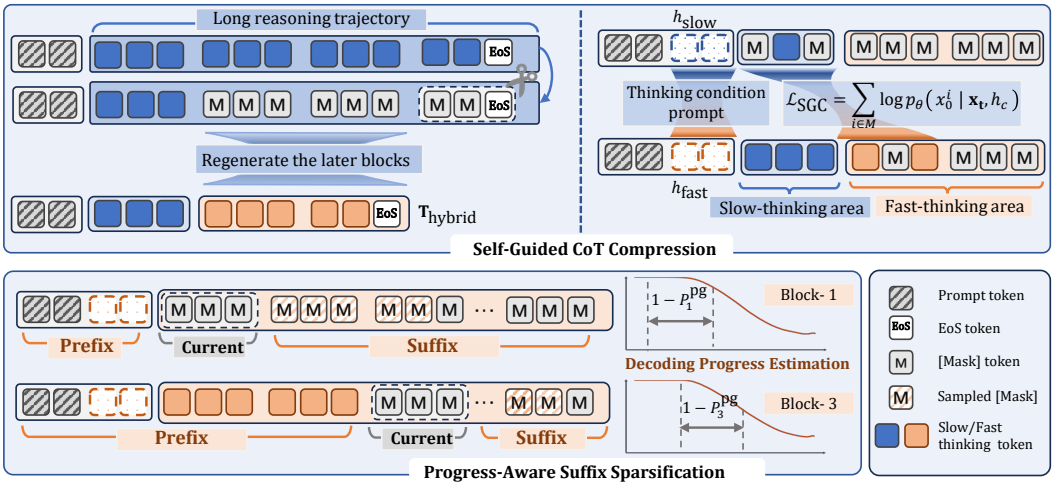

Figure 2: The overall pipeline of our proposed method. **(Top) Self-Guided CoT Compression (SGC):** We generate a long reasoning trajectory on a large canvas, truncate the later blocks, and regenerate them in a restricted canvas as the fast-thinking blocks. These hybrid-mode trajectories, paired with mode-specific prompts, are then used to fine-tune the dLLM via conditional SFT. **(Bottom) Progress-Aware Suffix Sparsification (PASS):** We adjust the number of sampled suffix tokens over the reasoning process based on decoding progress estimation P_b^{pg}, reducing redundant computation while maintaining essential context.

where t is sampled uniformly from $\mathcal{U}(0, 1)$, and $\mathbf{1}[\cdot]$ is the indicator function.

Inference of dLLMs. The sampling process of dLLMs begins by initializing a canvas with a fixed-length masked sequence. During generation, unmasked tokens remain unmodified, ensuring that the model focuses exclusively on refining the currently masked positions. Then, at each step, the model predicts distributions for all masked tokens on the canvas in parallel. It samples provisional tokens and applies a dynamic remasking strategy to determine which positions should be permanently unmasked and which should remain masked for further refinement in subsequent steps.

3.2 Self-Guided CoT Compression

Redundancy in Reasoning CoT. As introduced in Sec. 1, the oversized masked sequence often induces redundant reasoning trajectories, especially for simple problems. Here, we conduct a pilot study to further demonstrate this problem, which motivates our proposed Self-Guided CoT Compression (SGC) method. We collect GSM8K [12] problems that LLaDA-8B already solves correctly at canvas size 128, and measure accuracy and average response length as enlarging the canvas. As shown in Fig. 3, accuracy stays essentially, yet the average response length grows from 123 to 304 tokens, which is nearly 2.5× with a widening variance band. The result demonstrates that the CoT length is tightly entangled with the canvas size, meaning the model tends to take on more redundant reasoning steps as the canvas grows. To decouple the CoT length from the canvas capacity, we propose Self-Guided CoT Compression (SGC). This method utilizes the dLLM itself as an internal compressor to construct high-quality, variable-length CoT data.

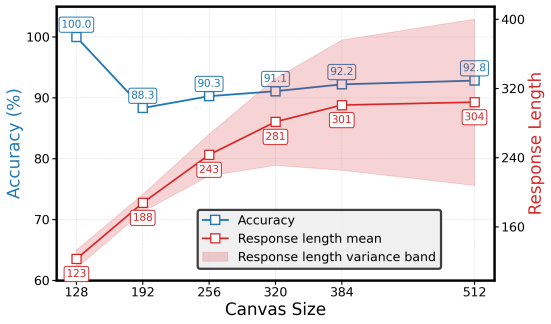

Figure 3: Illustration of entanglement between canvas size and response length on GSM8K. When enlarging the canvas, the average response length rises.

Data Construction via Self-Compression. The data construction process consists of two stages associated with different *thinking modes*, as illustrated in the left upper part of Fig. 2. Given an

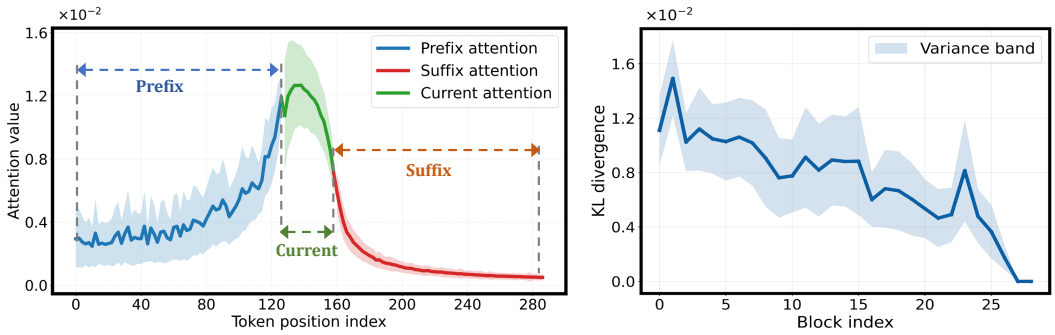

Figure 4: Pilot study on GSM8K. In (a), the adjacent suffix tokens contribute the most information gain to the current decoding block. In (b), the KL divergence between the predicted distributions with and without suffix context gradually decreases as decoding progresses, indicating diminishing gains as unmasking progresses.

input prompt (often including a question and exemplars), we first initialize a large canvas $\mathbf{C}_{\text{large}}$ to induce the model into a *slow-thinking mode*. In this setting, we encourage the dLLM to follow a comprehensive reasoning trajectory, including the correct final answer and an `<EOS>` token to terminate the generation. We denote the long reasoning trajectories as $\mathbf{T}_{\text{slow}}$. To obtain a compact CoT trajectory, we leverage the model's inherent self-compression capability. We randomly truncate the later reasoning blocks in $\mathbf{T}_{\text{slow}}$, and regenerate the remaining steps starting from these intermediate blocks. Crucially, we enforce a *fast-thinking mode* by restricting the available unmasking space to a narrowed canvas. Forced by the limited generation horizon, the dLLM acts as its own compressor and tries deriving the same correct answer in a much shorter and compact trajectory, denoted as $\mathbf{T}_{\text{hybrid}}$.

Conditional Supervised Fine-Tuning. To empower the model to generate compact CoTs even on a large canvas, we perform conditional supervised fine-tuning (SFT). Given a training dataset $\mathcal{D}$, we aggregate the trajectories $\mathbf{T}_{\text{hybrid}}$ by self-compression and filter out trajectories with incorrect answers to ensure data quality. Pairing the input prompts with their corresponding trajectories $\mathbf{T}_{\text{hybrid}}$, the dataset for conditional SFT is denoted as $\mathcal{D}_{\text{SGC}}$. To construct the masked sequence for training, we adopt the semi-autoregressive forward masking strategy as done in previous works [9, 44]. Specifically, the part tokens of the block currently being generated and the subsequent blocks are replaced with [MASK] token, while the previous blocks are unmasked and serve as context. This masking strategy effectively simulates the generation process of dLLMs. Besides, we assign distinct conditional prompts in the original instructions and guide the model to control the thinking mode while decoding the current block. Let h_{slow} and h_{fast} denote the prompt tokens indicating the *slow-thinking* and *fast-thinking* modes, respectively. The training objective leverages the standard negative log-likelihood loss on the masked positions, conditioned on these prompts:

$$\mathcal{L}_{\text{SGC}}(\theta) = -\mathbb{E}_{(\mathbf{x}^c)\sim\mathcal{D}_{\text{SGC}},t}\left[\frac{1}{t}\sum_{i=1}^{L}\mathbf{1}[x_t^i = \texttt{[MASK]}]\log p_\theta(x_0^i \mid \mathbf{x}_t, h_c)\right], \tag{3}$$

where $c \in \{\text{slow}, \text{fast}\}$, and $\mathbf{x}_0$ corresponds to the reasoning blocks from either the slow- or fast-thinking mode within the trajectory $\mathbf{T}_{\text{hybrid}}$. Through this conditional SFT, the model learns to associate the desired CoT reasoning modes with the corresponding conditional prompts.

During inference, we can explicitly specify h_{fast} to instruct the dLLM to yield concise CoTs even on a large canvas, thereby drastically saving parallel decoding steps.

3.3 Progress-Aware Suffix Sparsification (PASS)

Oversized canvas not only leads to redundant reasoning CoTs, but also introduces computational redundancy in the context utilization. With bidirectional unmasking, the model leverages not only the prefix context but also the unmasked suffix context to assist in the prediction of the current block. However, different from the unmasked prefix tokens that provide diverse reasoning context, the suffix tokens are [MASK] tokens with different position encodings, which provide relatively homogeneous context information.

Redundancy in Suffix Tokens as Context. To investigate inefficiency in suffix context utilization, we analyze the attention distribution of the current block across the entire input sequence on the GSM8K dataset. The attention distribution is shown in Fig. 4(a). We can see that compared to the prefix tokens that provide essential reasoning context, **the adjacent suffix tokens provide most information gains for the current decoding block**. Furthermore, we analyze the importance of suffix context across different decoding stages by a counterfactual intervention experiment. Specifically, for each decoding block, we quantify the Kullback-Leibler divergence between the model's predicted logit distributions for the current block with and without suffix context in Fig. 4(b). We find that **the context from the suffix tokens shows diminishing gains as decoding progresses.** We hypothesize that this is because, during the initial phase of generation, the decoding process is highly fragile and that suffix tokens are relatively important for preserving structural consistency. As unmasking proceeds, the model becomes increasingly confident, and the suffix tokens bring marginal information gain. Inspired by this observation, we believe that progressively adjusting the visible suffix context can reduce redundancy in the later decoding stages while preserving essential structure, thereby improving decoding efficiency.

Dynamic-length Suffix Window. In this work, we propose a progress-aware suffix sparsification strategy that dynamically shrinks the suffix window based on the estimated decoding progress. The key challenge lies in the reliable estimation of the current decoding progress. In practice, language models often terminate generation when predicting a special end-of-sequence token (<EOS>). This indicates that the model has implicitly learned to signal the completion of generation through the <EOS> token. Inspired by this, we use the model's confidence in predicting <EOS> over the masking tokens to estimate the decoding progress. Specifically, let $\mathcal{M}_b$ denote the set of masked token positions in the current decoding block b. We define the estimation of generation progress by taking the joint predicted probabilities of non-<EOS> token:

$$P_b^{\text{pg}} = 1 - \prod_{j \in \mathcal{M}_b} \Big(1 - p_\theta\Big(x_0^j = \texttt{<EOS>} \mid \mathbf{x}_t\Big)\Big). \tag{4}$$

Based on the decoding progress estimation, we dynamically adjust the suffix window for the corresponding decoding block. The window size is formulated as:

$$D_b = D^{\text{init}} \cdot \sigma(1 - P_b^{\text{pg}}), \tag{5}$$

where D^{init} is the hyper-parameter that bounds the maximum suffix window size, and $\sigma(\cdot)$ is the $\mathrm{sigmoid}$ function that maps the progress estimation to a scaling factor. According to the above formulation, during the fragile early stages, the model attends to more suffix tokens and absorbs rich structural cues. As decoding matures, the suffix horizon gradually shrinks towards a narrower window. Our PASS effectively reduces the computation overhead in each decoding step while maintaining the essential reasoning context.

4 Experiments

4.1 Experimental Setup

Baselines and Benchmarks. We evaluate the original dLLM under its official default inference setting (unmasking one token per step). We also compare our approach with recent methods that improve parallel decoding, including Fast-dLLM [46], KLASS [25], and dParallel [9]. We evaluate all methods on four reasoning benchmarks: GSM8K [12], MATH-500 [21], SVMAP [38], and Asdiv [35], which span different difficulty levels of mathematical reasoning. We report accuracy (Acc), the average number of decoding steps (Steps), and latency ($\mathrm{Lat.}$, measured as average runtime) to provide a comprehensive evaluation. To verify that our domain-specific fine-tuning does not lead to capability forgetting on other general QA tasks, we also compare the performance of our method with Fast-dLLM on several general benchmarks, including MBPP [5], MMLU [20], TruthfulQA [30](TQA), PIQA [7] and ARC-C [11].

Implementation Details. We evaluate our method on three representative open-source dLLMs: LLaDA-8B-Instruct (LLaDA-8B-Ins) [36], LLaDA-1.5 [54], and Dream-7B-Instruct [50] (Dream-7B-Ins). For semi-autoregressive masking, we set the block length $L_b = 32$. We construct training data from the GSM8K and MATH-500 training sets and sample 4K instances for training. We train for 4 epochs with a batch size of 128 and a learning rate of 1×10^{-6} on H20-GPUs. For complex

Table 1: Efficiency comparison with other inference acceleration methods on four reasoning benchmarks. We report accuracy (Acc), the average number of decoding steps (Steps), latency (Lat.).

Methods	GSM8K			MATH-500			SVMAP			Asdiv		
	Acc ↑	Steps ↓	Lat. ↓	Acc ↑	Steps ↓	Lat. ↓	Acc ↑	Steps ↓	Lat. ↓	Acc ↑	Steps ↓	Lat. ↓
LLaDA-8B-Ins												
Original [36] [NeurIPS2025]	82.8	512	11.1	39.0	512	12.4	88.7	512	7.3	84.5	512	7.3
Fast_dLLM [46] [ICLR2026]	82.3	91	2.8	39.0	148	4.8	88.7	65	1.9	83.6	65	1.9
KLASS [25] [NeurIPS2025]	80.9	129	4.4	37.2	201	7.0	85.7	95	3.3	83.8	96	3.2
dParallel [9] [ICLR2026]	79.1	37	1.2	37.2	67	2.1	83.5	26	0.9	80.3	27	0.9
Ours	81.0	39	1.1	38.2	65	1.9	86.4	25	0.8	81.4	26	0.8
LLaDA-1.5												
Original [50] [arXiv2025]	82.5	512	6.0	40.5	512	8.1	87.3	512	6.9	84.8	512	6.8
Fast_dLLM [46] [ICLR2026]	82.8	80	2.0	40.6	121	4.0	88.6	64	1.7	84.3	64	1.6
KLASS [25] [NeurIPS2025]	80.5	129	4.4	39.2	202	7.0	86.0	93	3.2	83.7	96	3.2
Ours	80.2	34	0.9	39.2	81	2.6	87.0	35	0.9	83.1	31	0.8
Dream-7B-Ins												
Original [50] [arXiv2025]	82.0	512	4.1	42.0	512	4.7	87.3	512	3.3	86.3	512	3.2
Fast_dLLM [46] [ICLR2026]	82.3	103	2.4	41.1	117	2.8	87.7	85	2.0	85.6	84	2.0
KLASS [25] [NeurIPS2025]	80.2	217	8.6	42.0	174	6.6	86.0	179	6.8	83.9	175	6.6
dParallel [9] [ICLR2026]	78.2	61	1.9	39.6	83	2.6	86.3	46	1.5	83.0	47	1.4
Ours	80.5	37	1.1	41.1	65	2.0	85.4	36	0.9	85.2	37	0.9

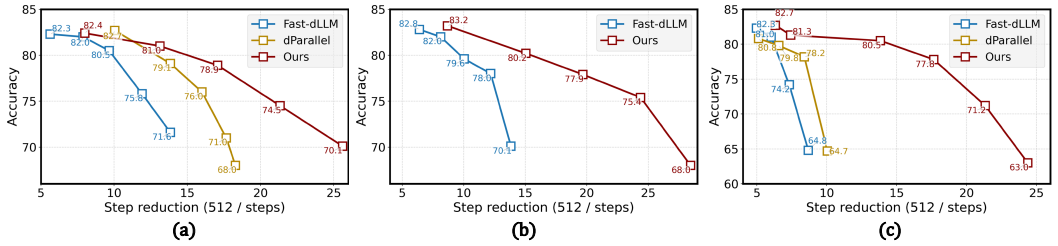

Figure 5: Comparison of speed-accuracy trade-off curves for Fast-dLLM, dParallel, and our method on GSM8K with LLaDA-8B-Ins (a), LLaDA-1.5 (b), and Dream-7B-Ins (c). The x-axis shows the reduction in decoding steps relative to the original setting (512 steps).

benchmarks such as MATH-500, we set the mask ratio to 50%; for other benchmarks, we set it to 90% to achieve stronger compression. During inference, following Fast-dLLM [46], we adopt a confidence-threshold semi-autoregressive remasking strategy and set the maximum generation length to 512 tokens. For suffix sparsification, we set the initial suffix window size D^{init} to 256 for a better balance between computation and performance.

4.2 Main Results

Results on Different Models. As shown in Tab. 1, our method consistently outperforms other methods in terms of the balance between speed and performance across three open-source dLLMs and different reasoning benchmarks. Specifically, on the GSM8K dataset, our method achieves approximately a $13\times$ reduction in decoding steps with only slight degradation to the original LLaDA model. As a result, the inference latency is reduced from $11.1s$ to $1.1s$, resulting in a $10\times$ speedup on the GSM8K dataset. Compared with Fast-dLLM and KLASS, which adopt more advanced parallel decoding strategies, our method achieves a more significant reduction in decoding steps and latency, while maintaining competitive accuracy. Besides, compared with dParallel which adopts a more aggressive parallel decoding by utilizing 100K data for consistency distillation, our method achieves a better balance between accuracy and efficiency. Moreover, our method is more training-efficient with only 4K training data. For complex benchmarks such as MATH-500, whose reasoning process requires more decoding steps, it is hard to construct effective self-compression data with the relatively weaker model capability. Our method still achieves a significant reduction in decoding steps and latency, while maintaining competitive accuracy compared to its counterparts.

Speed–Accuracy Trade-off. To further analyze the efficiency of different methods, we visualize the speed–accuracy trade-off curves of Fast-dLLM, dParallel and our method on the GSM8K dataset in Fig. 5. Specifically, we vary the confidence threshold to obtain different points on the trade-off curve. The horizontal axis represents the reduction factor of decoding steps compared to the original method (512 steps), and the vertical axis represents the accuracy. Our method consistently

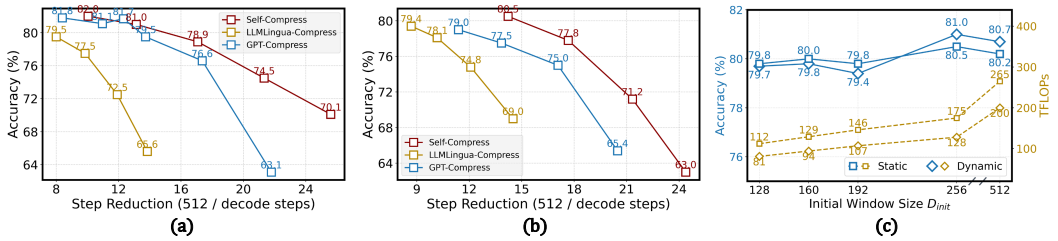

Figure 6: (a) and (b) show the comparison of different CoT compression strategies on the GSM8K dataset for LLaDA-8B-Ins and Dream-7B-Ins models, respectively. (c) shows the accuracy and compute (measured in TFLOPs) for different suffix window sizes under both static and our dynamic strategies on the GSM8K dataset with the LLaDA-8B-Ins model.

achieves a better trade-off between speed and accuracy across different dLLMs. Specifically, on the Dream-7B-Ins model, with an $8\times$ reduction in decoding steps, our method only suffers a minor drop in accuracy, while Fast-dLLM and dParallel suffer drops of 17.0% and 3.8% respectively, which strongly demonstrates our effectiveness in accelerating dLLMs inference.

4.3 Ablation Studies

Different CoT Compression Strategies. Instead of using external models, we propose utilizing the dLLM itself as the CoT compressor to construct the variable-length CoT trajectories for training. In order to verify the effectiveness of the self-compression strategy, we compare it with two other CoT compression strategies: `GPT-Compress` (use `GPT-5.2` for CoT compression) and `LLMLingua-Compress` [47] (use LLMLingua model for pruning the less important tokens) on GSM8K dataset for both LLaDA-8B-Ins and Dream-7B-Ins models, respectively. As shown in Fig. 6, we find that `LLMLingua-Compress` yields the worst speed-accuracy trade-off among the three methods. We attribute this to a weak capability to identify redundant tokens and to the semantic discontinuity introduced by token-level pruning. Though `GPT-Compress` generally provides stronger compression quality, the resulting CoT trajectories exhibit a larger distribution gap from the native dLLM trajectories, which reduces parallel decoding efficiency after fine-tuning. Our *Self-Compress* strategy strikes a better balance: it achieves effective CoT compression while retaining parallel decoding capability through self-distillation.

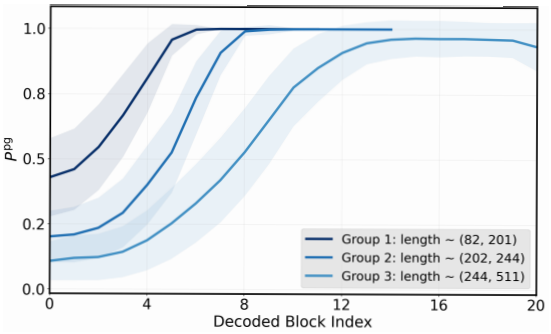

Figure 7: P_b^{pg} visualization across decoding blocks on different samples with different final generation lengths (measured in tokens) on the GSM8K dataset. Generally, with the decoding process, the metric P_b^{pg} gradually increases.

Different Suffix Sparsification Strategies. In Tab. 2, we compare three suffix sparsification strategies on GSM8K: *(i)* Uniform sampling, *(ii)* dPad with gaussian sampling, and *(iii)* our PASS strategy. Compared with dPad, the PASS strategy consistently reduces compute while maintaining or improving accuracy; for example, at N=256, compute drops from 172 to 128 TFLOPs ($\sim$25.6%) while achieving slightly better accuracy. PASS also outperforms Uniform sampling: at N=256, PASS reaches 81.0% accuracy with 128 TFLOPs, whereas Uniform sampling obtains 80.3% accuracy with 175 TFLOPs. These results indicate that the PASS strategy can achieve the best computational efficiency.

Table 2: Comparison of different suffix sparsification strategies on GSM8K dataset by LLaDA-8B-Ins model. We report accuracy and average compute (measured in TFLOPs) for different numbers of suffix tokens, N.

N	128	160	192	256	512
Uniform	78.9/112	79.8/129	79.8/146	**80.3/175**	80.5/255
dPad	79.3/108	79.4/124	79.7/139	**80.9/172**	80.5/255
PASS	79.7/81	79.8/94	79.4/107	**81.0/128**	79.8/200

Effectiveness of Decoding Progress Estimation. In Fig. 7, we visualize the decoding progress estimation P_{pg} across decoding blocks. We divide the samples into three groups based on their final generation lengths, and for each group, we compute the average P_{pg} at each decoding block. We can see that as the generation proceeds, P_{pg} exhibits a steady increase. Moreover, we find that samples with longer trajectories tend to have lower P_{pg} at the initial decoding stages, while shorter samples exhibit higher P_{pg}. These results suggest that P_{pg} can effectively reflect the progress of decoding, providing a reasonably accurate estimation of decoding progress.

Table 3: Hyper-parameter analysis by LLaDA-8B-Ins model on GSM8K dataset with different masking ratios used in the forward process during training.

Mask Ratio	0.5	0.7	0.8	0.9	1.0	original
Avg length	93	127	135	148	138	310
Steps	27	36	37	39	38	512
Acc	69.4	76.8	78.5	81.0	79.9	82.8

Mask Ratio Hyper-parameter Analysis. We analyze the impact of different mask ratios on model performance using the LLaDA-8B-Ins model on the GSM8K dataset. We compute the average response length (Avg length), the number of decoding steps, and the accuracy under different mask ratios in Tab. 3. Compared with the original model, our method substantially shortens response lengths and reduces the number of decoding steps. More aggressive compression (*e.g.*, a mask ratio of 0.5) yields shorter responses and fewer steps, but causes a larger accuracy drop. As the mask ratio increases, accuracy generally improves, while response length and the number of decoding steps increase overall. In this work, we set the mask ratio to 0.9 for a favorable balance.

Suffix Window Size Analysis. In Fig. 6(c), we visualize the accuracy and compute (measured in TFLOPs) of different window sizes for both *Static* (fixed suffix window size) and *Dynamic* (PASS) strategies on the GSM8K dataset with the LLaDA-8B-Ins model. We notice that for both *Static* and *Dynamic* strategies, the accuracy does not necessarily improve with the increase of suffix window size, which may be because more redundant context information that can interfere with the model's reasoning. For our *Dynamic* strategy, when setting the relatively small window size (*e.g.*, 128), the accuracy is relatively low. That is because when the window narrows with the decoding progress, the suffix context available to the model becomes extremely limited, which hinders the model's reasoning, leading to a drop in accuracy. With the increase of window size, the accuracy improves significantly and also achieves a better trade-off between accuracy and computation than *Static* strategy, by effectively utilizing the context information. In this work, we set D^{init} to 256 for a better balance between accuracy and computation on the GSM8K dataset.

Table 4: Comparison on general benchmarks with LLaDA-8B-Ins model.

Methods	**MBPP**	**MMLU**	**TQA**	**PIQA**	**ARC-C**
Original	38.1	59.3	31.2	76.0	52.8
Ours	37.8	59.0	30.1	76.2	53.6

Capabilities Analysis on General Benchmarks. In this work, we primarily use mathematical datasets for fine-tuning and evaluation. To verify that our method does not overfit to the mathematical domain and forget its capabilities on general question-answering (QA) tasks, we compare our fine-tuned model with the original model on several general benchmarks. As shown in Tab. 4, our method achieves comparable performance to the original model across all benchmarks, demonstrating that our domain-specific fine-tuning does not lead to significant capability forgetting on general QA tasks.

5 Conclusion

In this paper, we investigate dLLM acceleration from a trajectory-level perspective and show that oversized canvases induce two major inefficiencies: redundancy in CoT and redundancy in suffix context computation. To address these issues, we propose Beyond-the-Canvas dLLM (BC-dLLM), which integrates two complementary strategies: Self-Guided CoT Compression (SGC) to decouple reasoning length from canvas size, and Progress-Aware Suffix Sparsification (PASS) to dynamically reduce suffix computation as decoding progresses. By combining these strategies, the BC-dLLM framework substantially improves inference efficiency without sacrificing model capability. In future work, we will explore methods such as RLHF to further enhance the efficiency of dLLMs. Besides, we will also validate the effectiveness of BC-dLLM on longer and more diverse multimodal inputs, which is crucial for real-world applications.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.

[3] Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025.

[4] Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34:17981–17993, 2021.

[5] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[6] Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, et al. Llada2. 0: Scaling up diffusion language models to 100b. *arXiv preprint arXiv:2512.15745*, 2025.

[7] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

[8] Xinhua Chen, Sitao Huang, Cong Guo, Chiyue Wei, Yintao He, Jianyi Zhang, Hai Li, Yiran Chen, et al. Dpad: Efficient diffusion language models with suffix dropout. *arXiv preprint arXiv:2508.14148*, 2025.

[9] Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dparallel: Learnable parallel decoding for dllms. *arXiv preprint arXiv:2509.26488*, 2025.

[10] Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. *arXiv preprint arXiv:2510.06303*, 2025.

[11] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[13] Yingqian Cui, Pengfei He, Jingying Zeng, Hui Liu, Xianfeng Tang, Zhenwei Dai, Yan Han, Chen Luo, Jing Huang, Zhen Li, et al. Stepwise perplexity-guided refinement for efficient chain-of-thought reasoning in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18581–18597, 2025.

[14] Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, et al. Continuous diffusion for categorical data. *arXiv preprint arXiv:2211.15089*, 2022.

[15] Hengyu Fu, Baihe Huang, Virginia Adams, Charles Wang, Venkat Srinivasan, and Jiantao Jiao. From bits to rounds: Parallel decoding with exploration for diffusion language models. *arXiv preprint arXiv:2511.21103*, 2025.

[16] Zhujin Gao, Junliang Guo, Xu Tan, Yongxin Zhu, Fang Zhang, Jiang Bian, and Linli Xu. Empowering diffusion models on the embedding space for text generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4664–4683, 2024.

[17] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[18] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[19] Zhengfu He, Tianxiang Sun, Qiong Tang, Kuanning Wang, Xuan-Jing Huang, and Xipeng Qiu. Diffusionbert: Improving generative masked language models with diffusion models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 4521–4534, 2023.

[20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[21] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[22] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Xu Tang, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025.

[23] Daniel Israel, Guy Van den Broeck, and Aditya Grover. Accelerating diffusion llms via adaptive parallel decoding. *arXiv preprint arXiv:2506.00413*, 2025.

[24] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24312–24320, 2025.

[25] Seo Hyun Kim, Sunwoo Hong, Hojung Jung, Youngrok Park, and Se-Young Yun. Klass: Kl-guided fast inference in masked diffusion models. *arXiv preprint arXiv:2511.05664*, 2025.

[26] Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, et al. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.

[27] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. Llava-onevision: Easy visual task transfer. *Transactions on Machine Learning Research*.

[28] Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in neural information processing systems*, 35:4328–4343, 2022.

[29] Zeju Li, Jianyuan Zhong, Ziyang Zheng, Xiangyu Wen, Zhijian Xu, Yingying Cheng, Fan Zhang, and Qiang Xu. Compressing chain-of-thought in llms via step entropy. *arXiv preprint arXiv:2508.03346*, 2025.

[30] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 3214–3252, 2022.

[31] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.

[32] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.

[33] Xinyin Ma, Runpeng Yu, Gongfan Fang, and Xinchao Wang. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*, 2025.

[34] Yuxin Ma, Lun Du, Lanning Wei, Kun Chen, Qian Xu, Kangyu Wang, Guofeng Feng, Guoshan Lu, Lin Liu, Xiaojing Qi, et al. dinfer: An efficient inference framework for diffusion language models. *arXiv preprint arXiv:2510.08666*, 2025.

[35] Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th annual meeting of the Association for Computational Linguistics*, pages 975–984, 2020.

[36] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, JUN ZHOU, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

[37] Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *arXiv preprint arXiv:2406.03736*, 2024.

[38] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are nlp models really able to solve simple math word problems? In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 2080–2094, 2021.

[39] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of machine learning and systems*, 5:606–624, 2023.

[40] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[41] Yuerong Song, Xiaoran Liu, Ruixiao Li, Zhigeng Liu, Zengfeng Huang, Qipeng Guo, Ziwei He, and Xipeng Qiu. Sparse-dllm: Accelerating diffusion llms with dynamic cache eviction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33038–33046, 2026.

[42] Yuxuan Song, Zheng Zhang, Cheng Luo, Pengyang Gao, Fan Xia, Hao Luo, Zheng Li, Yuehang Yang, Hongli Yu, Xingwei Qu, et al. Seed diffusion: A large-scale diffusion language model with high-speed inference. *arXiv preprint arXiv:2508.02193*, 2025.

[43] Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[44] Yinjie Wang, Ling Yang, Bowen Li, Ye Tian, Ke Shen, and Mengdi Wang. Revolutionizing reinforcement learning framework for diffusion large language models. *arXiv preprint arXiv:2509.06949*, 2025.

[45] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[46] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025.

[47] Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3351–3363, 2025.

[48] Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, et al. Dream-coder 7b: An open diffusion language model for code. *arXiv preprint arXiv:2509.01142*, 2025.

[49] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[50] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

[51] Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning. *arXiv preprint arXiv:2505.16933*, 2025.

[52] Runpeng Yu, Xinyin Ma, and Xinchao Wang. Dimple: Discrete diffusion multimodal large language model with parallel decoding. *arXiv preprint arXiv:2505.16990*, 2025.

[53] Yangyang Zhong, Yanmei Gu, Zhengqing Zang, Xiaomeng Li, Yuqi Ding, Xibei Jia, Yuting Shen, Zhenzhong Lan, Liwang Zhu, Weiping Liu, et al. Parallelism and generation order in masked diffusion language models: Limits today, potential tomorrow. *arXiv preprint arXiv:2601.15593*, 2026.

[54] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.

[55] Zijian Zhu, Fei Ren, Zhanhong Tan, and Kaisheng Ma. Es-dllm: Efficient inference for diffusion large language models by early-skipping. *arXiv preprint arXiv:2603.10088*, 2026.

A Implementation Details

We evaluate the original dLLM under its official default inference setting (unmasking one token per step). We also compare our approach with recent methods that improve parallel decoding, including:
(1) Fast-dLLM [46]: adopts the confidence-threshold strategy to determine the unmasked tokens, and enables KV-cache on both prefix tokens and suffix tokens;
(2) KLASS [25]: leverages the model's internal dynamics in terms of token level KL divergence and confidence for parallel decoding;
(3) dParallel [9]: introduces a certainty-forcing distillation mechanism to improve the model's parallel decoding performance, and adopts an entropy-threshold strategy to determine the unmasked tokens during inference.
Following previous works [47, 24] in the autoregressive (AR) domain, we adopt a conditional training approach that trains language models with CoT data from different thinking modes simultaneously by inserting distinct prompt tokens in instructions. In this work, we use conditional prompt tokens: (i) *think detailedly and carefully* for slow mode, and (ii) *think compactly and concisely* for fast mode. In Tab. 5, we give details of the prompt templates on LLaDA-8B-Ins models.

Table 5: Details of prompt templates on LLaDA-8B-Ins models.

Methods	**Detailed Prompts**

Slow-thinking	`<|startoftext|><|start_header_id|>user<|end_header_id|>`
	`You need to put your final answer in \boxed {}. Please think detailedly and carefully.`
	`This is the problem:\n {{problem}}<|eot_id|><|startoftext|>`
	`<|start_header_id|>assistant<|end_header_id|>\n`

Fast-thinking	`<|startoftext|><|start_header_id|>user<|end_header_id|>`
	`You need to put your final answer in \boxed {}. Please think compactly and concisely.`
	`This is the problem:\n {{problem}}<|eot_id|><|startoftext|>`
	`<|start_header_id|>assistant<|end_header_id|>\n`

B Pseudo Code of Progress-Aware Suffix Sparsification

We provide the pseudo code for the inference process based on the proposed progress-aware suffix sparsification strategy in Algorithm 1. The algorithm takes as input the model M_θ, sequence length L, confidence threshold τ, block length L_b, and block number N_b. It outputs the generated sequence $\mathbf{x}$ after performing parallel decoding with dynamic suffix sparsification based on the estimated progress of decoding. During the decoding process, we also introduce an early stopping mechanism based on the presence of the EOS token. If the generated content contains an `<EOS>` token, we will terminate the decoding process early.

C Visualization of the Number of Sampled Suffix Tokens

In Fig. 8, we show the dynamic trend of the number of sampled suffix tokens for each block on GSM8K and MATH-500 datasets by the LLaDA-8B-Ins model. We can observe that on the GSM8K dataset, the model samples more suffix tokens in the first few blocks, but as the block increases, the number of sampled suffix tokens rapidly decreases, even approaching half of the initial number in subsequent blocks. On the MATH-500 dataset, the model shows a relatively stable decreasing trend in the number of sampled suffix tokens as the block increases, and it is only in the last few blocks that the number of sampled suffix tokens drops to a lower level. We hypothesize that for more complex problems, the model typically requires more chains of thought to reason and solve the problem step by step, and it is difficult to generate termination signals in the earlier blocks, thus sampling more suffix tokens. For simpler problems, the model may generate termination signals more quickly, resulting in fewer sampled suffix tokens in the earlier blocks. This reflects the different decoding dynamics of the model's reasoning process on datasets with varying difficulty levels.

Algorithm 1: Inference based on Progress-Aware Suffix Sparsification (PASS)

Input: model M_θ, sequence length L, confidence threshold τ, block length L_b, block number $N_\mathrm{b} = L/L_\mathrm{b}$
Output: Generated sequence $\mathbf{x}$
Initialize $\mathbf{x} \leftarrow [\mathrm{Prompt}; \mathrm{MASK}_{1:L}]$;
for $b \leftarrow 1$ **to** N_b **do**
  // Estimate the progress of decoding
  $\ell \leftarrow M_\theta(\mathbf{x}).\mathrm{logits}$;
  $p_\theta \leftarrow \mathrm{softmax}(\ell)$;
  $P_b^{\mathrm{pg}} = 1 - \prod_{j \in \mathcal{M}_b} \left(1 - p_\theta(\mathbf{x}^j = \texttt{<EOS>} \mid \mathbf{x}_t)\right)$;
  // Sparsify the suffix tokens in x
  $\mathbf{x}_{\mathrm{forward}} \leftarrow f_{\mathrm{PASS}}(\mathbf{x}, P_b^{\mathrm{pg}})$;
  for $t \leftarrow 1$ **to** L_b **do**
    $\ell \leftarrow M_\theta(\mathbf{x}_{\mathrm{forward}}).\mathrm{logits}$;
    $p_\theta \leftarrow \mathrm{softmax}(\ell)$;
    $c \leftarrow \max(p_\theta)$; // Get per-token confidence scores
    $\mathrm{high\_conf} \leftarrow (c > \tau)$;
    // Update sequence
    $\mathrm{unmask\_at\_indices}(\mathbf{x}, p_\theta, \mathrm{high\_conf})$;
  end
  // check EOS or Any mask token
  $\mathrm{Is\_masked} \leftarrow \mathrm{AnyMask}(\mathbf{x})$;
  $\mathrm{Is\_EOS} \leftarrow \mathrm{AnyEOS}(\mathbf{x})$;
  if Is_EOS or $\neg$ Is_masked **then**
    break;
  end
end
Return x

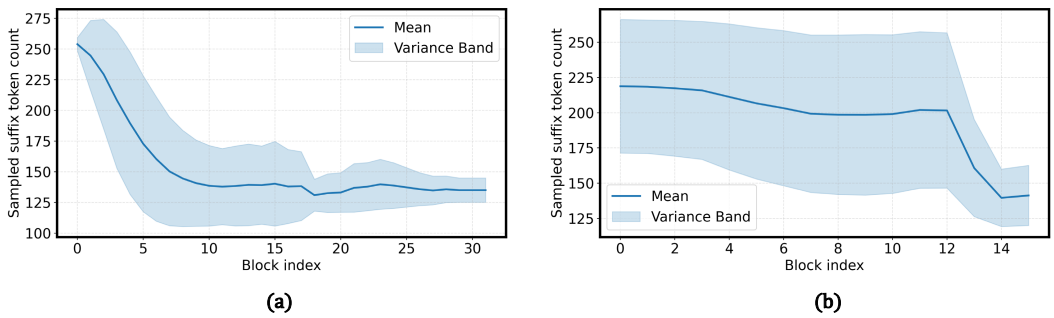

Figure 8: (a) and (b) respectively show the dynamic trend of the number of sampled suffix tokens for each block on GSM8K and MATH-500 datasets by the LLaDA-8B-Ins model.

D Ablation of the Required Compute

We further analyze the impact of introducing our proposed self-guided CoT compression (SGC) and progress-aware suffix sparsification (PASS) strategy on the average required compute (TFLOPs) for the LLaDA-8B-Ins model on GSM8K and MATH-500 datasets. As shown in Fig. 9, compared to the original dLLM, our method significantly reduces the required compute by approximately 60% on both datasets. Specifically, on the GSM8K dataset, the average required compute for the original dLLM is approximately 319 TFLOPs, which is reduced to about 175 TFLOPs after introducing our SGC strategy. With the further introduction of the PASS strategy, the average required compute is further reduced to around 128 TFLOPs. On the MATH-500 dataset, a similar trend is observed, with the average required compute decreasing from approximately 522 TFLOPs for the original dLLM to about 240 TFLOPs with SGC, and further down to around 218 TFLOPs with the addition of PASS.

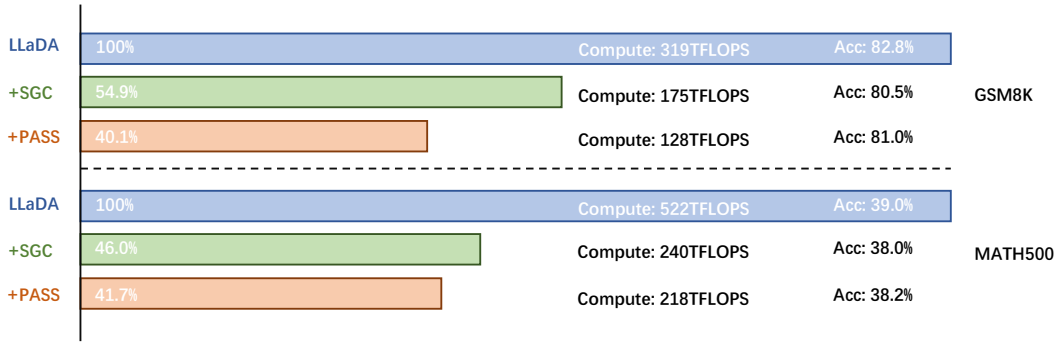

Figure 9: Comparison of the average required compute (TFLOPs) on GSM8K and MATH-500 datasets with the LLaDA-8B-Ins model.

Additionally, we note that on the MATH-500 dataset, the reduction in required compute brought by the SGC strategy is more significant, while the reduction brought by the PASS strategy is relatively smaller. This may be because for more complex problems, the model needs to sample more suffix tokens in more blocks, which is consistent with the trend we observed in Fig. 8.

E Case Studies

We further present a case study on the LLaDA-8B-Ins model to illustrate how our method influences generation behavior. When provided with a larger canvas, the model often tends to exhaust its full token budget, which results in verbose chains of thought and computationally inefficient outputs. In Fig. 10, we present a representative example on the GSM8K dataset, illustrating the verbose chain of thought generated by the LLaDA-8B-Ins model without CoT compression. This verbose chain of thought ($\sim$ 260 tokens) even leads to an incorrect final answer. In contrast, with our CoT compression method, the model generates a more concise chain of thought ($\sim$ 150 tokens) and ultimately produces the correct answer.

F Limitation and Future Work

Our method improves dLLM decoding efficiency from a trajectory-level perspective, but it still has several limitations. First, self-guided CoT compression relies on high-quality compact reasoning trajectories. When the model's capabilities are limited or the data quality is poor, the generated compressed chains of thought may fail to adequately cover the key information in the original chains of thought, leading to a decline in reasoning performance. In addition, we mainly focus on reasoning tasks with long chains of thought, and our CoT compression method may not bring significant efficiency improvements for tasks that do not require long chains of thought (*e.g.*, cloze tests, code completion). In the future, we plan to explore more advanced RLHF strategies to further enhance the quality of the generated compact reasoning trajectories and to investigate the effectiveness of our method on diverse multimodal reasoning tasks that require long chains of thought, such as visual question answering and video understanding.

G Broader Impacts

This work improves the efficiency of discrete diffusion LLMs by reducing unnecessary decoding steps and attention computation, which can lower inference cost and energy use and improve access for researchers with limited compute resources. Potential negative impacts should also be considered. When used as intended, faster generation can increase the scale of content production, including misleading or low-quality content. Under misuse, improved efficiency may lower the cost of harmful applications such as disinformation generation or automated abuse. Though these risks are not unique to our method, we encourage responsible use and urge the community to consider ethical implications when deploying more efficient LLMs.

LLaDA-8b-Instruct Compact Output Example

“question”: “Carol and Jennifer are sisters from Los Angeles who love collecting signatures from celebrities. During their summer break from school, the sisters spend every afternoon collecting signatures. After five weeks, Carol and Jennifer compare their autograph books, counting up the number of signatures each sister has collected. Carol has 20 signatures in her book, and Jennifer has 44. The sisters have three more weeks of summer vacation, and they decide they want to reach 100 signatures between them by the end of the summer. How many signatures do the sisters need to collect to reach their goal?”

“response”: "\nTo determine how many signatures Carol and Jennifer need to collect to reach their goal, we start by calculating the total number of signatures they currently have together.\n\nCarol has 20 signatures and Jennifer has 44 signatures. Therefore, the total number of signatures they currently have is:\n\\[20 + 44 = 64 \\]\n\nTheir goal is to reach 100 signatures between them. To find out how many more signatures they need, we subtract the number of signatures they currently have from their goal:\n\\[100 - 64 = 36 \\]\n\nThus, they need to collect \\(\\boxed{36}\\) signatures to reach their goal.”

"response_length": 150

(a)

LLaDA-8b-Instruct Original Output Example

“question”: “Carol and Jennifer are sisters from Los Angeles who love collecting signatures from celebrities. During their summer break from school, the sisters spend every afternoon collecting signatures. After five weeks, Carol and Jennifer compare their autograph books, counting up the number of signatures each sister has collected. Carol has 20 signatures in her book, and Jennifer has 44. The sisters have three more weeks of summer vacation, and they decide they want to reach 100 signatures between them by the end of the summer. How many signatures do the sisters need to collect to reach their goal?”

“response”: “\nTo determine how many signatures Carol and Jennifer need to collect to reach their goal of 100 signatures between them by the end of the summer, we can follow these steps:\n\n1. Calculate the total number of signatures they currently have.\n2. Determine how many more signatures they need to reach their goal.\n3. Calculate the number of signatures they need to collect in the remaining three weeks.\n\nFirst, let's calculate the total number of signatures they currently have:\n- Carol has 20 signatures.\n- Jennifer has 44 signatures.\n- Total current signatures = \\(20 + 44 = 64\\).\n\nNext, we determine how many more signatures they need to reach their goal:\n- Their goal is 100 signatures.\n- Signatures needed = \\(100 - 64 = 36\\).\n\nFinally, we calculate the number of signatures they need to collect in the remaining three weeks:\n- They have 3 weeks left.\n- Signatures needed per week = \\(\\frac{36}{3} = 12\\).\n\nTherefore, the sisters need to collect \\(\\boxed{12}\\) signatures each week to reach their goal.”

"response_length": 260

(b)

Figure 10: (a) and (b) respectively show the compact output after compressing the CoT and the original detailed output on the GSM8K dataset by the LLaDA-8B-Ins model.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: In Sec.1, we make the claims to reflect the paper's contributions and scope.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: In the Sec. F, we discuss the limitations of the work.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

Justification: The paper does not include theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: We provide implementation details in Sec. 4.1 and the Appendix for reproducibility.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We will consider releasing the data and code once the paper is accepted.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: We provide details about the experimental setting (*e.g.*, hyper-parameters) in Sec. 4.1 and the Appendix for reproducibility.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We do not report error bars due to limited computing resources.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We provide the information on the computer resources in Sec. 4.1.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research conducted in the paper conform with the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The paper discusses societal impacts of the work in the Appendix G.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper poses no such risks.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: The paper properly cites the existing assets.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper does not release new assets.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: Some LLMs are acted as our baselines in our experiments. We clarify their settings and usage following open source benchmark.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.